



FACE RECOGNITION ATTENDANCE SYSTEM USING AI



PROJECT GUIDE

DR.M.VAMSIKRISHNA, M.Tech.,Ph.D.

Pydi Rupa Bhaskar

23A91F0004

Design Document

Introduction

This document outlines the design of **FaceCheck**, an AI-driven system for automating attendance tracking using **real-time face recognition**. The system employs computer vision techniques like **facial landmark detection**, **deep learning embeddings**, and **cosine similarity matching** to identify individuals and log attendance securely. The following sections detail the system architecture, workflows, and database design, ensuring scalability and privacy compliance.

Design Goals

1. **Accuracy:** Achieve >98% recognition accuracy using DeepFace embeddings.
2. **Real-Time Performance:** Process video feeds at 15+ FPS with <1s latency.
3. **Scalability:** Support 1,000+ users and adapt to varying lighting/angles.
4. **Privacy:** Store facial embeddings (not raw images) with AES-256 encryption.
5. **Portability:** Deploy on edge devices (Raspberry Pi) or cloud servers.

Functional Requirements

1. **Input:**
 - **Camera Feed:** Real-time video from HD webcam or IP camera (720p resolution, 30 FPS).
 - **User Enrollment:** Admin uploads user photos or captures live via webcam.
 2. **Processing:**
 - **Face Detection:** Locate faces using OpenCV Haar Cascade or DNN.
 - **Face Recognition:**
 1. Generate 128D embeddings via DeepFace's VGGFace model.
 2. Match against stored embeddings using cosine similarity (threshold > 0.6).
 - **Attendance Rules:** First recognition → Mark present; ignore duplicates within 5 minutes.
 3. **Output:**
 - **Visual Feedback:** Green bounding box + name overlay for recognized faces; red box for unknowns.
 - **Attendance Reports:** Daily summaries (CSV/PDF) with filters (date, department).
 4. **Logging:**
 - Store attendance records with timestamp, user ID, and recognition confidence.
-

System Architecture

1. High-Level Architecture:

- **Input Layer:** Captures video via camera module.
- **Processing Layer:**
 - **Detection:** Preprocess frames → Detect faces → Align landmarks.
 - **Recognition:** Generate embeddings → Match with database.
- **Output Layer:** Log attendance → Generate alerts/reports.

2. System Components:

- **Camera Module:** USB/IP camera integration (OpenCV VideoCapture).
- **Recognition Module:** DeepFace-based identity verification.
- **Database Module:** SQLite (edge) or PostgreSQL (cloud) for user/attendance data.
- **Reporting Module:** Pandas/Matplotlib for CSV/PDF reports.

Detailed Design

1. Face Detection:

- **Input:** Raw video frames.
- **Method:** OpenCV Haar Cascade (CPU) or ResNet-10 SSD (GPU).
- **Output:** Cropped face regions with bounding boxes.

2. Face Recognition:

- **Embedding Generation:** DeepFace's VGGFace model (128D vectors).
- **Matching:** Cosine similarity > 0.6 (configurable threshold).

3. Attendance Logic:

- **Check-In:** First match → Log as "Present".
- **Anomaly Handling:** Flag unrecognized faces for admin review.

Algorithms and Methods

Component	Algorithm	Library
Face Detection	Haar Cascade/ResNet-10 SSD	OpenCV
Face Alignment	Affine Transformation	dlib
Embedding Generation	VGGFace (SE-ResNet-50)	DeepFace
Similarity Matching	Cosine Similarity	SciKit-Learn

Component	Algorithm	Library
Liveness Detection	Eye Blink Detection (EAR)	MediaPipe

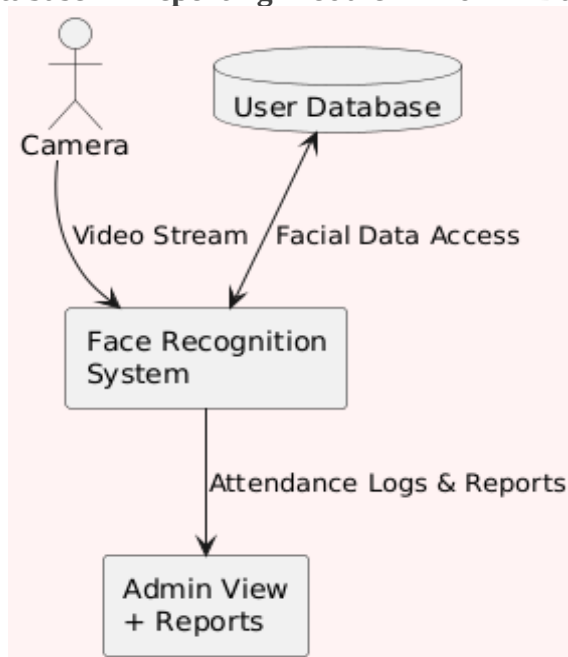
Context Flow Diagram

Entities:

- **Camera:** Captures facial video.
- **Admin:** Manages user database and views reports.
- **Database:** Stores embeddings and attendance logs.
- **Alert System:** Flags unrecognized faces.

Flow:

1. **Camera** → **Face Detection** → **Recognition** → **Database Update**.
2. **Database** → **Reporting Module** → **Admin Dashboard**.



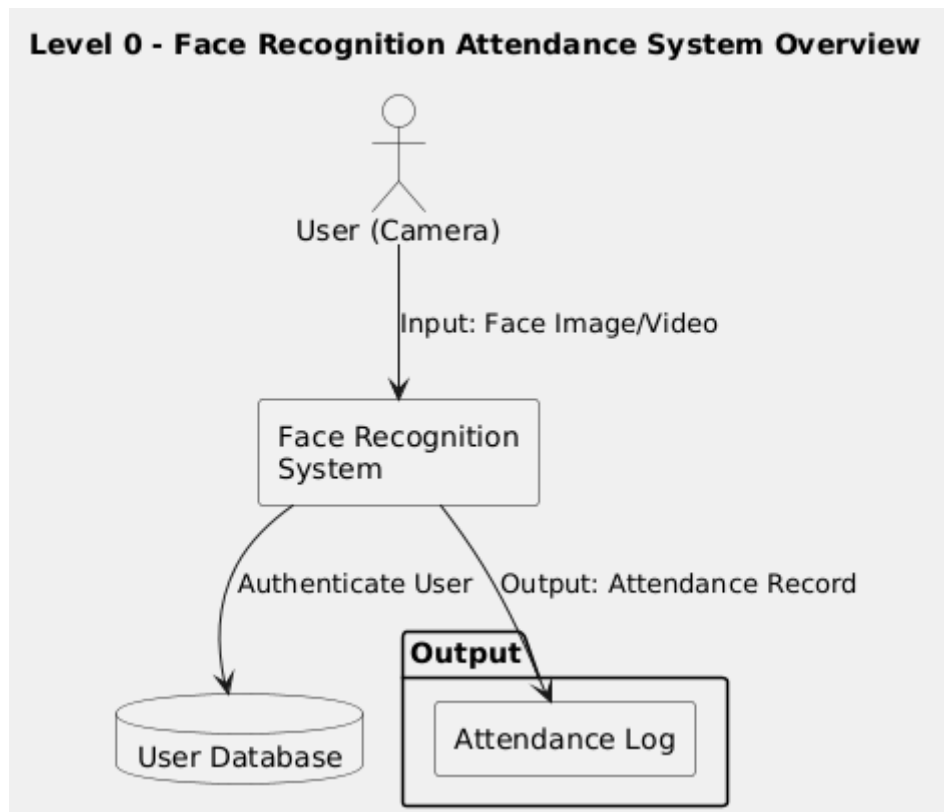
+Data Flow Diagram (DFD)

The **Data Flow Diagram (DFD)** represents the data movement within the system. It is hierarchical and moves from high-level (Level 0) to detailed (Level 1 and Level 2).

Level 0: Overview of the System

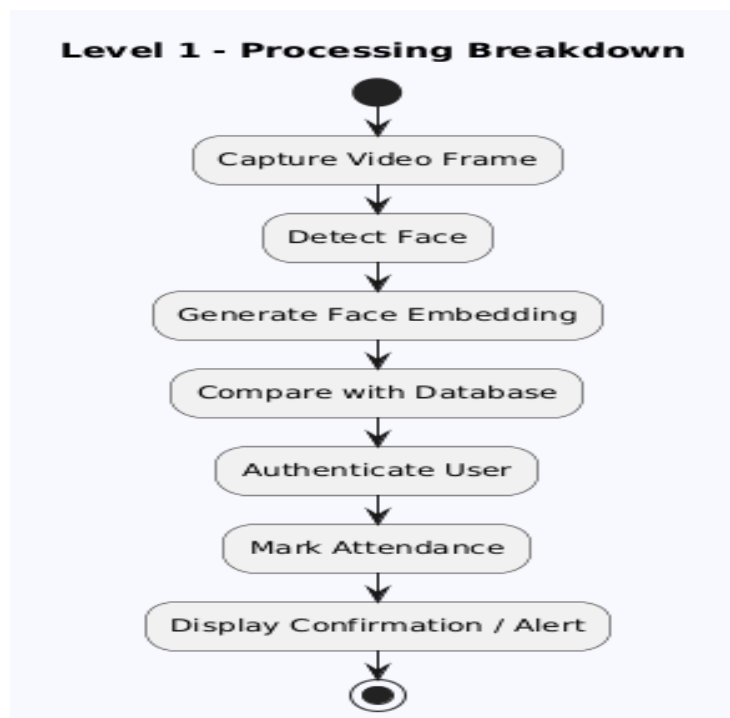
- **Input:** Video stream from the camera.
- **Processing:** Drowsiness detection (EAR, MAR, head pose).

- **Output:** Alerts and event logs.



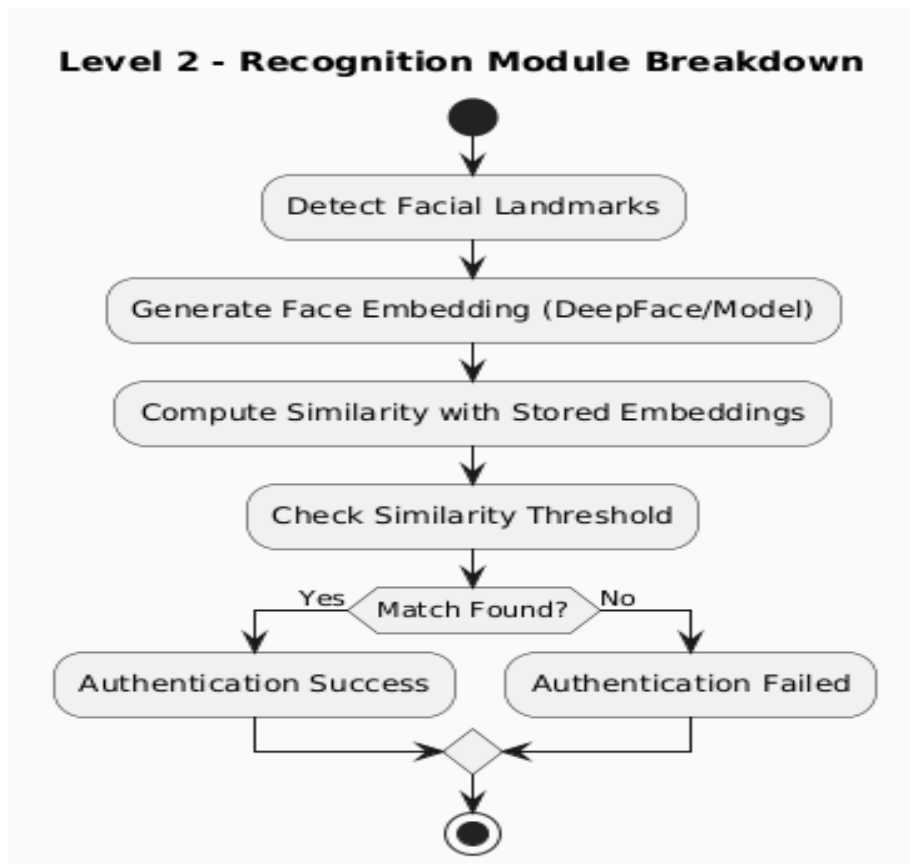
Level 1: Processing Breakdown

1. **Input from Camera:**
 - Video frames are captured.
2. **Detection Module:**
 - EAR, MAR, and head pose are calculated.
3. **Decision Module:**
 - Combines detection results to classify drowsiness.
4. **Outputs:**
 - Alerts generated.
 - Data logged for analysis



Level 2 (Recognition Module):

- Detect landmarks → Generate embedding → Compute similarity → Threshold check.



ER Diagram: Defines the database structure for FaceCheck.

Entities:

1. **User** :(UserID-PK, Name, Department, 128D Embedding).

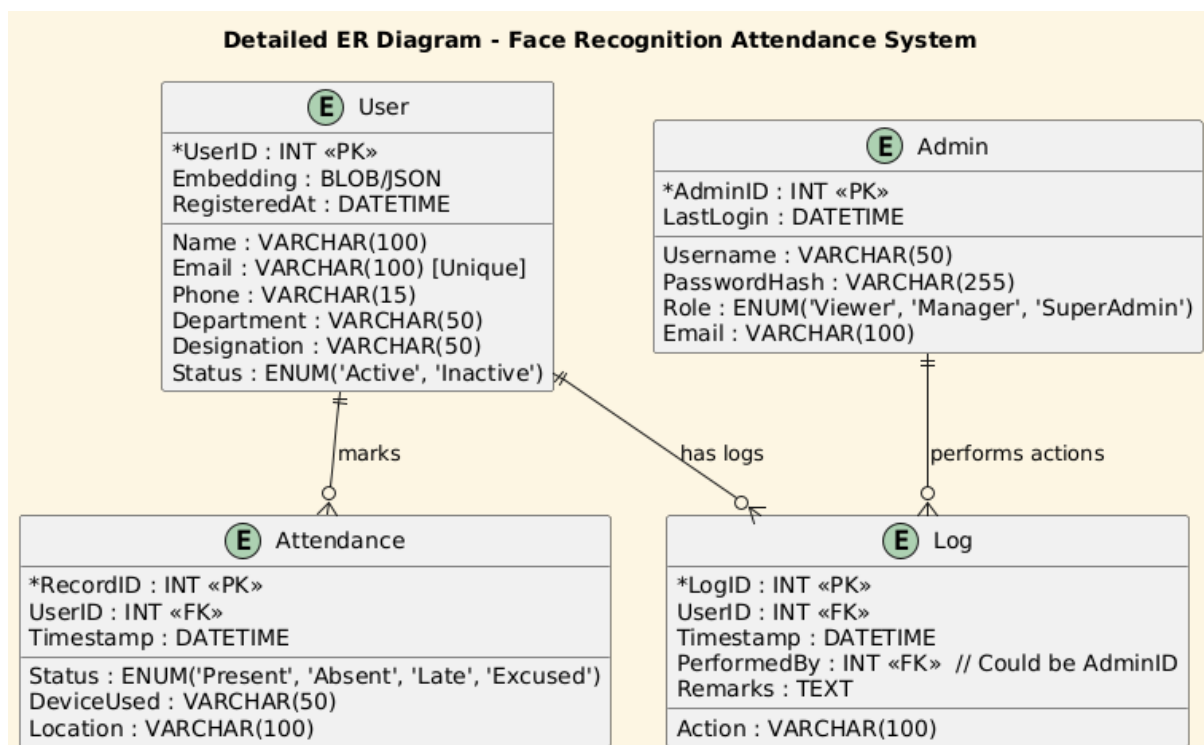
2. **Attendance**:(RecordID-PK, UserID-FK, Timestamp, Status).

Relationship: One-to-many (User→Attendance) – each user linked to multiple attendance records.

Keys: UserID uniquely identifies users; foreign key in Attendance ties records to users.

Design: Normalized tables minimize redundancy; embeddings ensure privacy.

Purpose: Enables efficient querying of attendance logs by UserID/date.



Non-Functional Requirements

1. **Performance:**
 - Minimum 10 FPS real-time processing.
 2. **Security:**
 - Encrypt face embeddings and user credentials.
 3. **Compatibility:**
 - Supports Windows/Linux; Deployable on Raspberry Pi.
 4. **Maintainability:**
 - Easy to upgrade to new face recognition models.
 5. **Privacy:**
 - GDPR-compliant data storage options.
-

Hardware & Software Requirements

Hardware

- HD Webcam
- CPU: ≥ 2.5 GHz (Quad Core)
- RAM: ≥ 4 GB

Software

- **Language:** Python 3.8+
 - **Libraries:** OpenCV, DeepFace, NumPy, Pandas, SQLite/MySQL
 - **OS:** Windows/Linux
 - **Deployment:** PyInstaller/Docker
-

Testing Plan

- **Unit Testing:** Test face detection, recognition individually.
 - **Integration Testing:** Test pipeline from camera to attendance logging.
 - **Performance Testing:** Measure FPS and accuracy under different lighting and angles.
 - **User Testing:** Verify with known and unknown faces.
-

Deployment Plan

1. Train face data and store embeddings.
2. Package application as executable using PyInstaller.
3. Deploy on local machine or Raspberry Pi.
4. Configure camera and test in real-world setting.

Conclusion

The **Face Recognition Attendance System** introduces a smarter, faster, and more secure way to manage attendance through advanced AI. By leveraging **DeepFace** and **OpenCV**, the system ensures high **recognition accuracy**, **real-time monitoring**, and **privacy compliance**, making it ideal for educational institutes, organizations, and secured facilities.